

Customer Order Insights & Delivery Tracker

Week 5 – Deploy & Automate with Azure DevOps

1. Objective

The purpose of Week 5 is to automate the execution of the Customer Order project scripts using **Azure DevOps Pipelines**. This enables continuous integration and deployment (CI/CD) so that the Python scripts run automatically whenever changes are pushed to the repository.

Deliverables:

1. Azure DevOps YAML pipeline file (azure-pipelines.yml)
2. Pipeline execution log or console output

2. Prerequisites

Before creating the pipeline, the following are required:

1. **Azure DevOps Account** – Active subscription with access to DevOps Projects.
2. **Code Repository** – Project code stored in Azure Repos or linked GitHub repository.
3. **Project Files to be pushed to repository root:**
 - project_pipeline.py (main Python script prepared in Colab, downloaded, and uploaded here)
 - requirements.txt (list of dependencies: pandas, numpy, pyspark)
 - CSV datasets (orders.csv, customers.csv, delivery_status.csv)
 - azure-pipelines.yml (pipeline definition file)

Repository Structure:

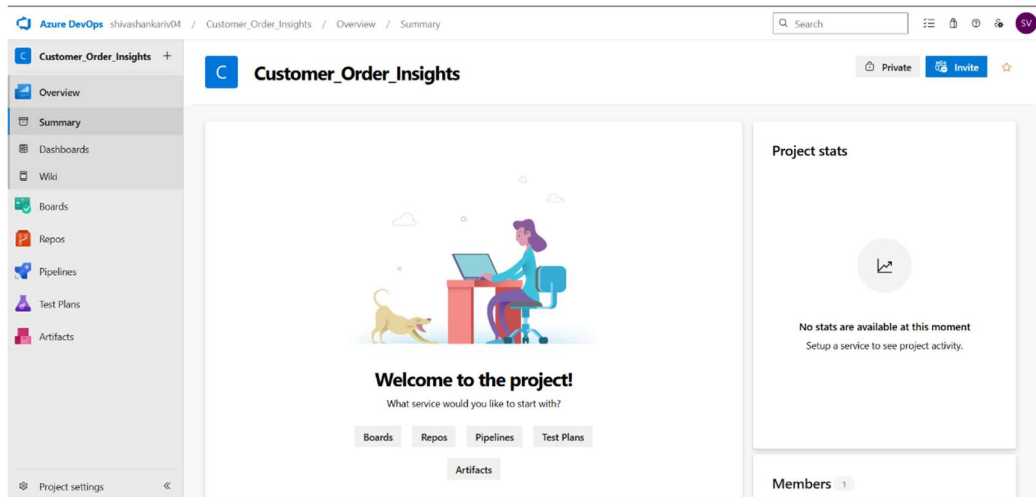
/ (root)

```
|— Customer_Order_pipeline.py
|— requirements.txt
|— orders.csv
|— customers.csv
|— delivery_status.csv
└— azure-pipelines.yml
```

3. Step-by-Step Process

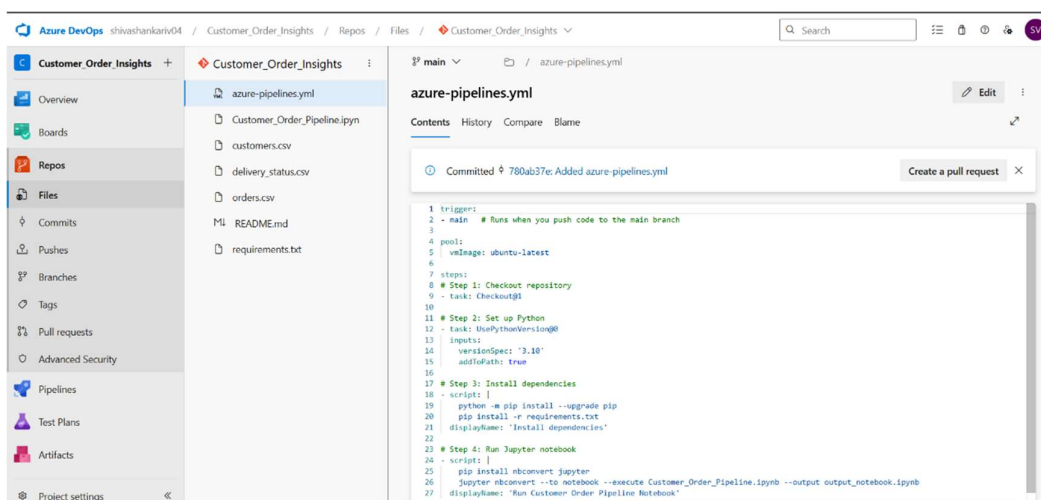
Step 1: Create Azure DevOps Project

1. Sign in to **Azure DevOps**.
2. Create a new project (e.g., CustomerOrder_Insights).
3. Connect repository (Azure Repos).



Step 2: Prepare and Push Repository

1. Upload all project files (.ipynb / .py, .csv, requirements.txt, azure-pipelines.yml) to the repository root.
2. Confirm that the directory structure matches the prerequisites so the pipeline can locate files correctly.
3. Ensure that the azure-pipelines.yml file is placed at the root of the repository, as Azure DevOps automatically looks for it there when defining pipelines.

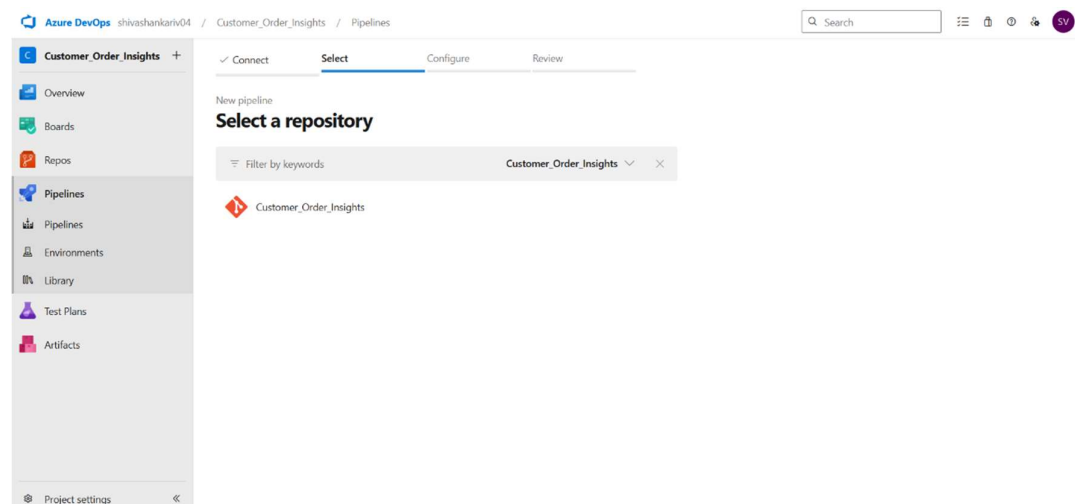
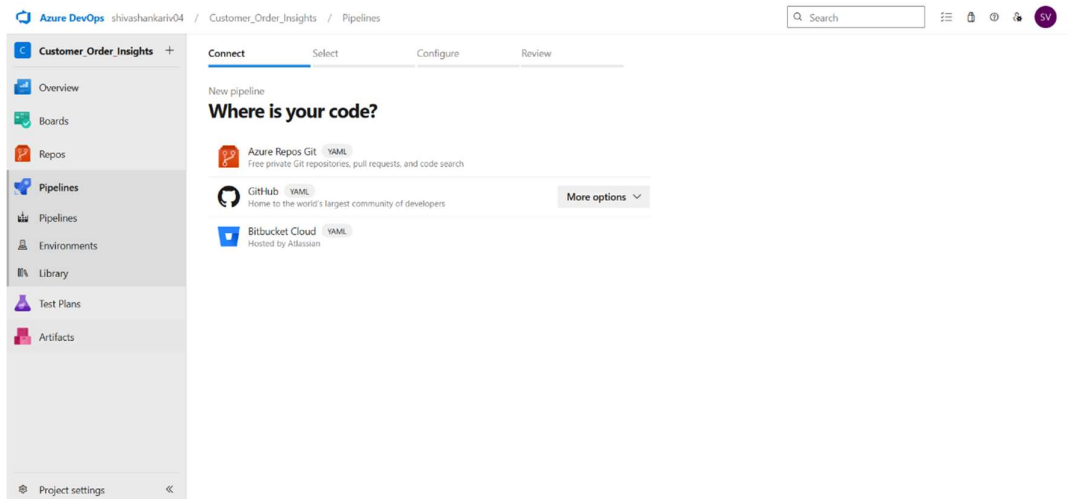


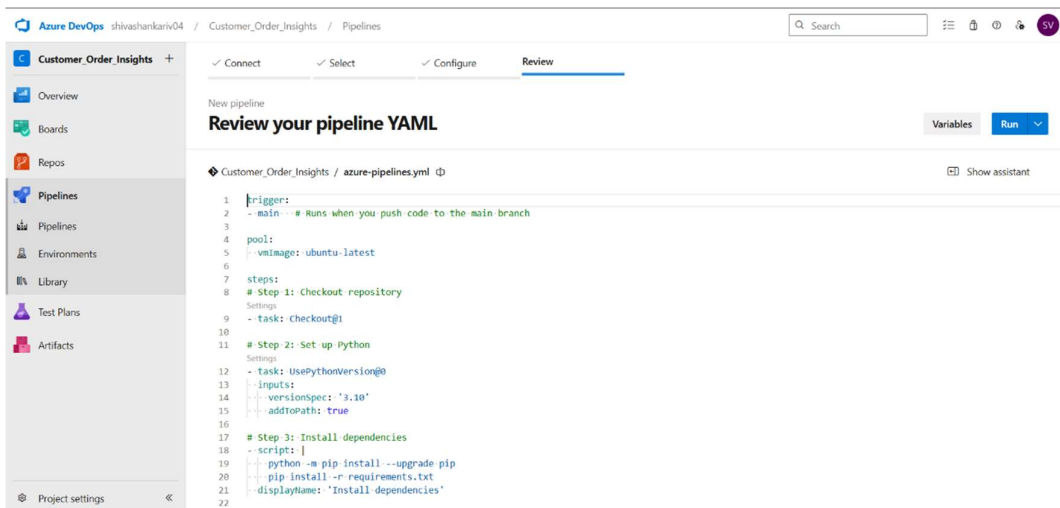
Step 3: Define the Pipeline (YAML)

Create and commit the pipeline configuration file. This YAML file contains the build and execution steps that automate installation of dependencies and execution of the file.

Step 4: Run the Pipeline

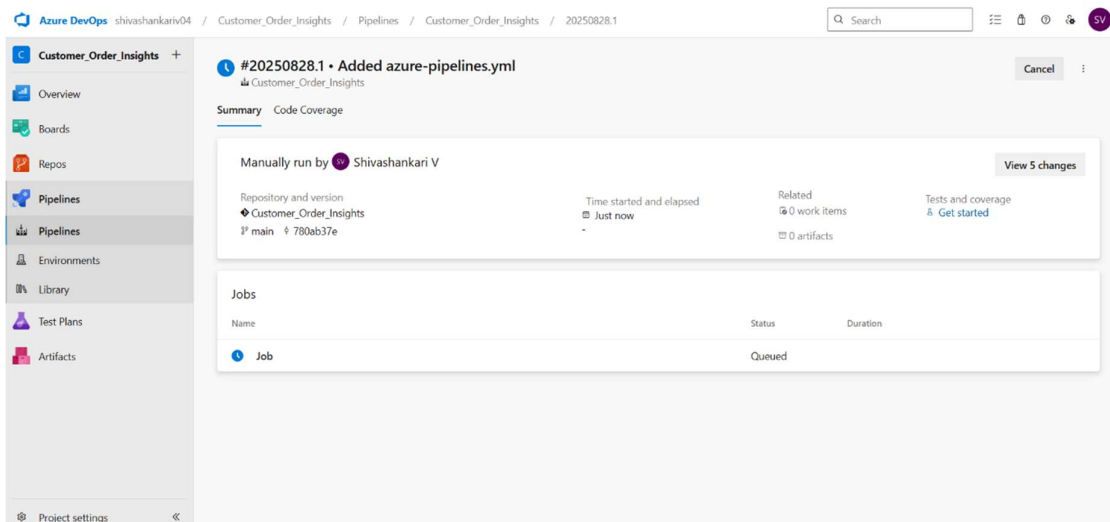
1. Go to **Pipelines > New Pipeline**.
2. Select repository and point to YAML file (azure-pipelines.yml).
3. Run manually or wait for an auto-trigger on commit.
4. Monitor execution logs in the Azure DevOps console.





Step 5: Validate & Collect Deliverables

1. Confirm the pipeline runs successfully.
2. Save the execution log which should display:
 - Dependency installation steps.
 - Script outputs (delay detection, top delayed customers, delivery status summary).
3. Document results as proof of successful automation.



4. Example Execution Log (Sample)

Starting: Run Customer Order Pipeline

=====

Collecting pandas

Collecting numpy

Collecting pyspark

...

>>> Running project_pipeline.py

Orders Processed: 10

Delayed Orders: 3

Most Common Issue: Pending Deliveries

Top Delayed Customer: anita sharma

=====

Pipeline completed successfully

5. Conclusion

In Week 5, the Customer Order project was automated with **Azure DevOps Pipelines**. The YAML file ensured:

- Dependencies installed consistently
- Scripts executed automatically on commit
- Logs generated for monitoring

This approach integrates CI/CD practices, ensuring repeatable and reliable execution of analytics tasks.